iNeuBytes — Virtual Internship Program (VIIP)

Artificial Intelligence — Course ID: AIINB20726

Major Project: Full-Stack AI Application

Objective: Develop and deploy a full-stack AI web application that integrates a Machine Learning or Natural Language Processing model and works as a live web service.

What you are building

You must build a working web application that anyone can open in a browser, use easily, and receive an AI-powered result. The application should allow a user to:

- Open a public web link
- Enter an input or select an option
- Submit the input
- Receive a meaningful ML/NLP-based result
- Use the application without needing your help

By the end of the project, you should be able to share a public URL where your application works live on the internet, not only on your local laptop.

Interns choose ONE option

Option	Project Type	What It Should Do
Option A	Recommendation System	The user enters something they like (movie, product, song, or course) and the app returns relevant recommendations.
Option B	Personalized Chatbot	The user types a message, and the app returns a relevant, context-aware response using an NLP-based model.

Project Requirements

Dataset

- Dataset must be between 50 MB and 500 MB
- A larger dataset is not always better — free hosting platforms have memory/storage limits, so the model should be optimized for deployment
- The 50 MB minimum applies only to the dataset, not the final deployed model, which should be as lightweight as possible

Backend (Flask)

The backend is the main engine of the application. It must:

- Load the trained model only once when the server starts
- Accept user input through an API request
- Process the input using the ML/NLP model
- Return a clean JSON response
- Handle errors without crashing

Required backend endpoints

Endpoint	Purpose
/recommend or /respond	Accepts user input and returns the model output as JSON. Use /recommend for recommendation systems and /respond for chatbots.
/health	Returns a simple status message such as "OK" to confirm that the server is running.

Error handling requirement

The backend must not crash when the user sends incorrect input. For example, if the input is empty or the request format is wrong, the API should return a clear error message instead of a 500 server error page.

Security requirement

- Do not commit secrets to GitHub
- Never upload: API keys, database passwords, private tokens, secret configuration files
- Use environment variables for sensitive information

Frontend requirement

The frontend is what the user sees and interacts with. It must include: a clear input box, a submit button, a visible result area, simple instructions for the user, and proper display of the result without showing raw JSON. The design does not need to be advanced, but it must be clean, usable, and understandable for a non-technical person.

API testing requirement: Postman

Backend APIs must be tested using Postman. For each endpoint, document: endpoint URL, request method, input sent, response received, response time, and at least one error case.

Deployment requirement

The application must be deployed on a public platform such as:

- Render
- Railway
- PythonAnywhere

Deployment performance guidelines

ML models can exceed the memory limits of free hosting platforms. To avoid deployment failure:

- Keep the model lightweight
- Use a smaller trained model where possible
- Apply model compression or quantization if needed
- Avoid bundling very large model files inside the GitHub repository
- Load large files from external storage if required

Response time target

The application should ideally return a response within 3 to 5 seconds per request on free hosting. Some free hosting platforms may sleep after inactivity — if your app has a slow first load, mention the expected cold-start time clearly in your documentation.

Evaluation Criteria

The project will be considered complete when:

- The live application link works from any device
- A non-technical user can use the application without instructions
- The model returns sensible and relevant results
- The output is not random, broken, or meaningless

- Empty or incorrect input is handled properly
- The backend does not crash during normal usage
- The GitHub repository can be run from a clean clone
- The README clearly explains setup, usage, testing, and deployment

Final Deliverables

You must submit:

- Live deployed application link
- GitHub repository link
- Dataset details and source
- Trained model or model loading instructions
- Frontend source code
- Flask backend source code
- requirements.txt
- README file
- Postman testing screenshots or documentation
- Screenshots of the working application
- Short explanation of the model used and how it works

Completion standard

A successful project should prove that you can build, connect, test, and deploy a complete AI application. The final output should not be only a notebook or a local demo — it must be a usable full-stack AI web application available through a public link.